

Bài 4: CÁC PHƯƠNG THỨC CỦA LỚP ĐỐI TƯỢNG

Mục tiêu

Về kiến thức:

- Khái niệm các hàm thành phần, xác định các hàm thành phần, cú pháp định nghĩa các hàm thành phần, nạp chồng các hàm thành phần, gọi các hàm thành phần, nguyên lý truyền thông điệp giữa các đối tượng

Về kỹ năng:

Giúp sinh viên xây dựng được chương trình HĐT vận dụng cú pháp khai báo các hàm thành phần để định nghĩa các hàm thành phần của lớp giải quyết bài toán ở mức vừa và phức tạp..

4.1. Định nghĩa hàm thành phần và cơ chế nạp chồng, viết đè trong

Java

4.1.1. Các hàm thành phần của lớp

Hành vi của các đối tượng thuộc một lớp được xác định bởi các hàm thành phần của lớp đó, hành vi thể hiện “vai trò” của đối tượng đó trong hệ thống.

Là các hàm, thủ tục thực hiện các phép xử lý trên các biến thành phần.

* Khai báo

*<Phạm_vi> <Kiểu_giá_tri> <tên_hàm>(< danh sách các tham số hình thức>)
<các_mệnh_đề_throws> { // Thân hàm }*

Phạm_vi:

- *public*: Cho phép truy cập mọi nơi trong hệ thống
- *private*: Chỉ được truy xuất (gọi) bên trong lớp chứa nó
- *protected*: lớp đó và lớp con của nó được truy xuất
- *final*: Không cho phép nạp chồng và ghi đè
- *static*: Nó có ý nghĩa tương tự như với thuộc tính *static*, thông thường chỉ những phương thức chung và không dùng cho một đối tượng cụ thể thì khi đó phương thức đó nó mới là phương thức tĩnh (*static*).

- *abstract*: Phương thức trừu tượng (là phương thức “mẫu”, nó chỉ nói lên là nó làm được gì mà không mô tả nó làm như thế nào), do đó nó không cài đặt nội dung ở lớp khai báo nó

- *native*: dùng cho phương thức khi cài đặt phụ thuộc môi trường trong một ngôn ngữ khác, như C hay hợp ngữ.

- *synchronized*: dùng trong lập trình đa luồng, nhằm ngăn các tác động của các phương thức khác của các luồng khác khi phương thức đang được thực hiện.

- Giá trị mặc định là *public*.

<Kiểu trả lại> có thể là kiểu nguyên thủy, kiểu lớp hoặc không có giá trị trả lại (kiểu *void*).

<Danh sách tham biến hình thức> bao gồm dãy các tham biến (kiểu và tên) phân cách với nhau bởi dấu phẩy.

*** Sử dụng phương thức:**

- Nếu phương thức không phải là static thì cần phải tạo ra một đối tượng cụ thể và gọi phương thức thông qua tên đối tượng như cấu trúc sau:

<tên đối tượng>.<tên_hàm>(< danh sách các tham số hiện thời >)

Khai báo đối tượng: <tên lớp> <tên đối tượng> ;

Khởi tạo giá trị thuộc tính cho đối tượng khi sử dụng Toán tử tạo lập (constructor) tự định nghĩa :

<tên đối tượng> = new <tên lớp> (<danh sách tham số hiện thời >) ;

Khai báo và khởi tạo đối tượng giá trị:

<tên lớp> <tên đối tượng> = new <tên lớp> (<danh sách tham số hiện thời >) ;

- Nếu phương thức là static thì gọi thông qua tên lớp hoặc thông qua tên đối tượng (nên gọi các phương thức tĩnh thông qua tên lớp)

<tên_lớp>.<tên_hàm_tĩnh> (<danh sách tham số hiện thời >)

- Các tham số trong các lời gọi hàm cung cấp cách thức trao đổi thông tin giữa đối tượng gửi và đối tượng nhận thông điệp.

- Số các tham biến của danh sách hiện thời phải bằng số các tham biến của danh sách tham biến hình thức.
- Kiểu của các tham biến hiện thời phải tương thích với kiểu của tham biến hình thức tương ứng.

Kiểu của tham biến hiện thời	Giá trị được truyền
Các kiểu nguyên thủy	Giá trị kiểu nguyên thủy
Kiểu lớp (class)	Giá trị tham chiếu
Kiểu mảng (array)	Giá trị tham chiếu

Ví dụ1: Tập Demo.java

```

public class Demo {
    public static void main(String args[]) {
        int i = 5;
        if (i == 0) {
            return;
        }
        output(checkValue(args.length));
    }
    static void output(int value) {
        System.out.println(value);
    }
    static int checkValue(int i) {
        if (i > 3) {
            return 1;
        } else {
            return 2;
        }
    }
}

```

Ví dụ2: Tập Demo2.java

```

public class Demo2 {
    public static void main(String args[]) {
        int i = 5;
        Demo2 d = new Demo2();
        if (i == 0) {
            return;
        }
    }
}

```

```

        output(d.checkValue(args.length));
    }
    static void output(int value) {
        System.out.println(value);
    }
    public int checkValue(int i) {
        if (i > 3) {
            return 1;
        } else {
            return 2;
        }
    }
}

```

*** Phương thức và thuộc tính static:** Phương thức và thuộc tính độc lập với đối tượng, có thể sử dụng mà không cần thông qua đối tượng, Phương thức tĩnh: không sử dụng được thuộc tính và phương thức “thông thường” (non-static) mà nó chỉ được gọi đến các phương thức và thuộc tính tĩnh khác

4.1.2 Truyền tham số và gọi phương thức(hàm)

Truyền các giá trị kiểu nguyên thủy

Trong Java, khi gọi một phương thức và truyền các giá trị nguyên thủy cho phương thức, được gọi là truyền tham trị. Việc thay đổi giá trị chỉ có hiệu lực trong phương thức được gọi, không có hiệu lực bên ngoài phương thức.

Các tham số có kiểu dữ liệu nguyên thủy là byte, short, int, long, float, double, boolean, char.

Ví dụ: *Truyền các giá trị nguyên thủy*

```

class KhachHang1 { // Lớp khách hàng
    public static void main(String[] arg) {
        HangSX banh = new HangSX(); // Tạo ra một đối tượng
        int giaBan = 20;
        double tien = banh.tinh(10, giaBan);
        System.out.println("Gia ban: " + giaBan); // giaBan không đổi
        System.out.println("Tien ban duoc : " + tien);
    }
}

```

```

    }
}
// Lớp Hãng sản xuất
class HangSX {
    double tinh(int num, double gia) {
        gia = gia / 2;
        return num * gia; // Thay đổi gia nhưng không ảnh hưởng tới giaBan,
        số tiền bị thay đổi theo
    }
}

```

Truyền các giá trị tham chiếu đối tượng.

Trong Java, khi gọi một phương thức và truyền một tham chiếu cho phương thức, được gọi là truyền tham chiếu. Việc thay đổi giá trị của biến tham chiếu bên trong phương thức làm thay đổi giá trị của nó.

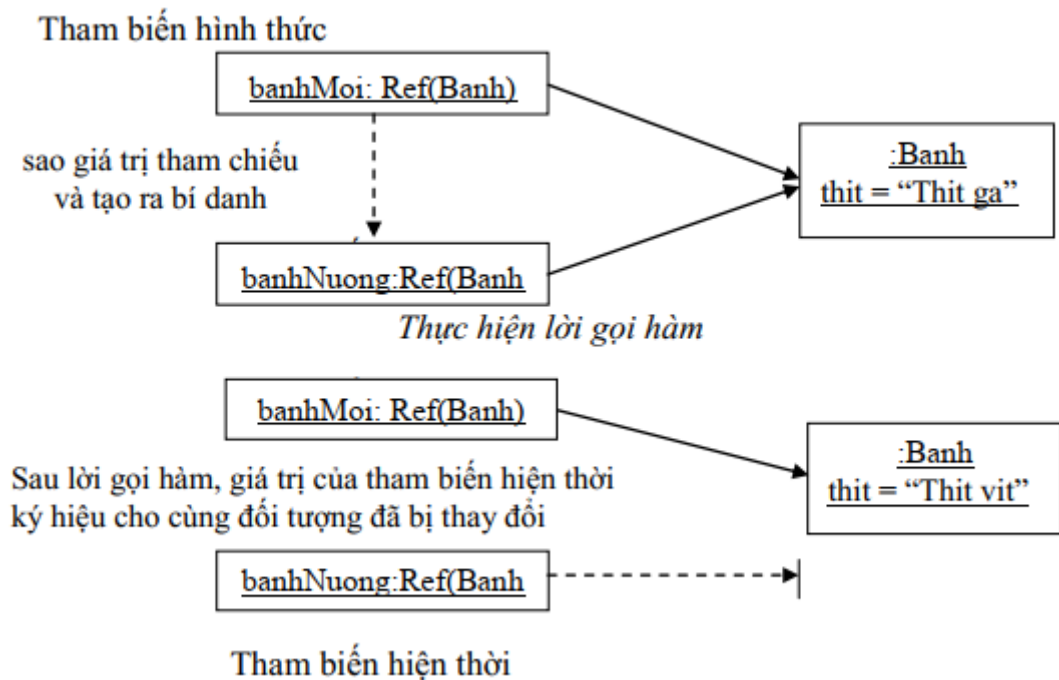
Trong Java, tất các phương thức có tham số là biến có kiểu là các lớp (class) đều là kiểu tham chiếu

```

//KhachHang2.java
class KhachHang2 { // Lớp khách hàng
    public static void main(String[] arg) {
        Banh banhMoi = new Banh(); // Tạo ra một đối tượng (1)
        System.out.println("Nhoi thit vao banh truoc khi nuong:" +
        banhMoi.thit);
        nuong(banhMoi); // (2)
        System.out.println("Thit cua banh sau khi nuong:" + banhMoi.thit);
    }
    public static void nuong(Banh banhNuong) { // (3)
        banhNuong.thit = "Thit vit
        "; // đổi nhân thành thịt vịt
        banhNuong = null; // (4)
    }
}
class Banh { // Lớp Banh (5)

```

```
String thit = "Thịt gà"
"; // Qui định của hãng làm nhân bánh bằng thịt gà
}
```



Truyền tham chiếu: đối với đối tượng thì nội dung của tham chiếu (LValue) được copy lên stack.

Truyền đối số trong dòng lệnh

```
class Pass {
    public static void main(String parameters[]) {
        System.out.println("This is what the main method received");
        System.out.println(parameters[0]);
        System.out.println(parameters[1]);
        System.out.println(parameters[2]);
    }
}
```

Biên dịch chương trình: **javac PassArgument.java**

Thực thi chương trình với dòng lệnh: **java PassArgument A 123 B1**

Sẽ thu được trên màn hình kết quả:

This is what the main method received

A

123

Các tham biến final

Tham biến loại này được gọi là biến cuối “trắng”, nghĩa là nó không được khởi tạo giá trị (là trắng) cho đến khi nó được gán một trị nào đó và khi đã được gán trị thì giá trị đó là cuối cùng, không thay đổi được.

Ví dụ *Sử dụng tham biến final*

```
//KhachHang3.java
class KhachHang3 { // Lớp khách hàng
    public static void main(String[] arg) {
        HangSX banh = new HangSX(); // Tạo ra 1 đối tượng
        int giaBan = 20;
        double tien = banh.tinh(10, giaBan);
        System.out.println("Gia ban: " + giaBan); // giaBan không đổi
        System.out.println("Tien ban duoc : " + tien);
    }
}

// Lớp Hãng sản xuất
class HangSX {
    double tinh(int num, final double gia) { // (1)
        gia = gia / 2.0; // (2)
        return num * gia; // Thay đổi gia nhưng không ảnh hưởng tới giaBan,
        số tiền vẫn bị thay đổi theo
    }
}
```

4.1.3. Cơ chế nạp chồng, viết đè trong Java

Nạp chồng

Nạp chồng là cho phép sử dụng cùng tên hàm nhưng định nghĩa nhiều nội dung thực hiện khác nhau.

Các hàm nạp chồng phải khác nhau về danh sách tham biến (khác nhau về kiểu tham biến hoặc số lượng tham biến hay là có định danh khác nhau)

Ví dụ: Xây dựng lớp các hình học, trong đó xây dựng các hàm nạp chồng tính diện tích

```

public class HinhHoc {
    static int dienTich(int a) {
        System.out.println("Dien tich hinh vuong: ");
        return a * a;
    }
    static int dienTich(int a, int b) {
        System.out.println("Dien tich chu nhat: ");
        return a * b;
    }
    static double dienTich(int a, double b) {
        System.out.println("Dien tich tron: ");
        return a * a * b;
    }
    public static void main(String[] arg) {
        int a = 3;
        int b = 4;
        double c = 3.14;
        System.out.println(" " + dienTich(a));
        System.out.println(" " + dienTich(a, c));
        System.out.println(" " + dienTich(a, b));
    }
}

```

JDK API đã xây dựng rất nhiều hàm được nạp chồng.

Lưu ý là danh sách tham biến của các hàm nạp chồng phải khác nhau về số lượng hoặc về thứ tự các kiểu của các tham biến.

Ví dụ:

Hàm	Định danh
public void methodA(int a, double b){/* ...*/}	methodA(int, double)
public int methodA(int a){return a}	methodA(int)

public int methodA(){return 1} // (3)	methodA()
public long methodA(double a,int b){return a*b}	methodA(double, int)
public long methodA(int a, double b){return a} //NoOK	methodA(int, double) không đúng do định danh giống (1")

Viết chồng (viết đè)

Một lớp con có thể viết đè, thay đổi nội dung thực hiện của những hàm thừa kế từ một lớp cha gọi là viết đè các hàm thành phần.

Khi những hàm này được gọi để thực thi đối với những đối tượng của lớp con thì nội dung thực hiện được định nghĩa ở lớp con sẽ được thực thi.

Ví dụ:

- Xây dựng lớp Meo có hàm thành phần speak()
- Xây dựng lớp MeoCon kế thừa lớp Meo và viết đè hàm speak()
- Gọi hàm speak() của lớp MeoCon
- Gọi hàm speak() của lớp Meo đối với đối tượng thuộc lớp MeoCon

```
class Meo{
    protected static void speak(){
        System.out.print("Meo Meo!"); }
}
class MeoCon extends Meo{
    protected static void speak(){
        System.out.print("Miu Miu!"); }
    public static void main(String []arg){
        Meo meo=new Meo();
        MeoCon meocon=new MeoCon();
        MeoCon.speak();
        System.out.println();
        meo.speak();
        System.out.println(); }
}
```

*** Lưu ý:**

- Định nghĩa mới của hàm viết đè phải có cùng định danh (tên gọi và danh sách tham biến) và cùng kiểu trả lại giá trị.
- Định nghĩa mới của hàm viết đè trong lớp con chỉ có thể xác định tất cả hoặc tập con các lớp ngoại lệ được kể ra trong mệnh đề cho qua ngoại lệ
- Các hàm final, static không được phép viết đè.
Phân biệt rõ hai cơ chế viết đè và nạp chồng là khác nhau trong Java.
- Viết đè yêu cầu cùng định danh hàm (cùng tên gọi, cùng danh sách tham số) và cùng kiểu trả lại đã được định nghĩa tại lớp cha.
- Nạp chồng yêu cầu khác nhau về định danh, nhưng giống nhau về tên gọi của hàm, vì thế chúng sẽ khác nhau về số lượng, kiểu, hay thứ tự của các tham biến.
- Hàm có thể nạp chồng ở trong cùng lớp hoặc ở các lớp con cháu.
- Từ những lớp con khi muốn gọi tới các hàm ở lớp cha mà bị viết đè thì phải gọi qua toán tử đại diện cho lớp cha, đó là super(). Đối với hàm nạp chồng thì lại không cần như thế. Lời gọi hàm nạp chồng được xác định thông qua danh sách các đối số hiện thời so sánh với đối số hình thức để xác định nội dung tương ứng.

4.1.4. Nạp chồng các toán tử tạo lập

Giống như các hàm thành phần, các toán tử tạo lập có thể nạp chồng với nhiều nội dung thực hiện khác nhau.

Khi tạo lập đối tượng sử dụng toán tử tạo lập nào thì toán tử đó được gọi.

Ví dụ:

```
class BongDen{  
    // Biến thành phần (1)  
    private int soWatts;  
    private boolean batTat;  
    private String viTri;  
    // Định nghĩa toán tử tạo lập số 1 (2)  
    BongDen(){  
        this(40, true);  
        System.out.println("Toán tử số 1"); }  
}
```

```

// Định nghĩa toán tử tạo lập số 2 (3)
BongDen(int w, boolean s){
    this(w, s, "XX");
    System.out.println("Toán tử số 2"); }

//Định nghĩa toán tử tạo lập số 3 (4)
BongDen(int soWatts, boolean batTat, String viTri){
    this.soWatts = soWatts;
    this.batTat = batTat;
    this.viTri = new String(viTri);
    System.out.println("Toán tử số 3"); }
}

class DenTuyp extends BongDen {
    private int doDai;
    private int mau;
    DenTuyp(int leng, int colo){
        this(leng,colo,100,true,"Chua biet");
    }
    DenTuyp(int leng, int colo, int soWatt, boolean bt, String noi){
        super(soWatt, bt, noi);
        this.doDai = leng;
        this.mau = colo;
    }
}

public class NhaKho{
    public static void main(String args[]){
        System.out.println("Tao ra bong đèn tuyp");
        DenTuyp d = new DenTuyp(20, 5);
    }
}

```

4.1.5. Cơ chế che bóng của các biến

Về nguyên tắc, một lớp con không được phép viết đè các biến thành phần của lớp cha, nhưng có thể che khuất chúng tương tự như biến cục bộ trong lớp con.

Ví dụ Viết đè và nạp chồng các hàm thành phần

```

// KhachHang.java

import java.io.*;

class Den {
    protected String loaiHoaDon = "Hoa don nho: "; // (1)
    protected double docHoaDon(int giaDien) throws Exception { // (2)
        double soGio = 10.0, hoaDonNho = giaDien * soGio;
        System.out.println(loaiHoaDon + hoaDonNho);
        return hoaDonNho;
    }
}

class DenTuyp extends Den {
    public String loaiHoaDon = "Hoa don lon: "; // Bị che bóng (3)
    public double docHoaDon(int giaDien) throws Exception { // Viết đè hàm
(4)
        double soGio = 100.0;
        hoaDonLon = giaDien * soGio;
        System.out.println(loaiHoaDon + hoaDonLon);
        return hoaDonLon;
    }
    public double docHoaDon() {
        System.out.println("Khong co hoa don!");
        return 0.0;
    }
}

public class KhachHang {
    public static void main(String args[]) throws Exception { // (6)
        DenTuyp den1 = new DenTuyp(); // (7)
        Den den2 = den1; // (8)
        Den den3 = new Den(); // (9)
        // Gọi các hàm đã viết đè
        den1.docHoaDon(1000); // (10)
        den2.docHoaDon(1000); // (11)
    }
}

```

```

        den3.docHoaDon(1000); // (12)
// Truy nhập tới các biến thành phần đã bị viết đè (bị che bóng)
        System.out.println(den1.loaiHoaDon); // (13)
        System.out.println(den2.loaiHoaDon); // (14)
        System.out.println(den3.loaiHoaDon); // (15)
// Gọi các hàm nạp chồng
        den1.docHoaDon();
    }
}

```

Kết quả thực hiện của chương trình KháchHang:

Hóa đơn lớn: 100000.00

Hóa đơn lớn: 100000.00

Hóa đơn nhỏ: 10000.00

Hóa đơn lớn:

Hóa đơn nhỏ:

Hóa đơn nhỏ:

Không có Hóa đơn!

4.2. Các thuộc tính kiểm soát truy nhập các thành phần của lớp

Khi xây dựng một lớp ta có thể hạn chế sự truy cập đến các thành viên của lớp, từ một đối tượng khác. Ta tóm tắt qua bảng sau:

Từ khoá	Truy cập trong chính lớp đó	Truy cập trong lớp con cùng gói	Truy cập trong lớp con khác gói	Truy cập trong lớp khác cùng gói	Truy cập trong lớp khác khác gói
private	X	-	-	-	-
protected	X	X	X	X	-
public	X	X	X	X	X
default	X	X	-	X	-

Trong bảng trên thì X thể hiện cho sự truy cập hợp lệ còn – thể hiện không thể truy cập vào thành phần này.

***Ví dụ:** Các thành phần private

Các thành viên private chỉ có thể sử dụng bên trong lớp, ta không thể truy cập các thành viên private từ bên ngoài lớp này.

```
class Alpha {  
    private int iamprivate;  
    private void privateMethod() {  
        System.out.println("privateMethod");  
    }  
}  
  
class Beta {  
    void accessMethod() {  
        Alpha a = new Alpha();  
        a.iamprivate = 10; // không hợp lệ  
        a.privateMethod(); // không hợp lệ  
    }  
}
```

* Ví dụ: Các thành phần public

Các thành viên public có thể truy cập từ bất cứ đâu, ta sẽ xem ví dụ sau:

```
Package Greek;  
  
public class Alpha {  
    public int iampublic;  
    public void publicMethod() {  
        System.out.println("publicMethod");  
    }  
}  
  
package Roman;  
import Greek.*;  
  
class Beta {  
    void accessMethod() {  
        Alpha a = new Alpha();  
        a.iampublic = 10; // hợp lệ  
        a.publicMethod(); // hợp lệ  
    }  
}
```

*Ví dụ các thành phần có mức truy xuất gói:

Khi ta khai báo các thành viên mà không sử dụng một trong các từ public, private, protected thì java mặc định thành viên đó có mức truy cập gói.

Ví dụ:

```
package Greek;
class Alpha {
    int iampackage;
    void packageMethod() {
        System.out.println("packageMethod");
    }
}
package Greek;
class Beta {
    void accessMethod() {
        Alpha a = new Alpha();
        a.iampackage = 10;// hợp lệ
        a.packageMethod();// hợp lệ
    }
}
```

4.3. Bài giải mẫu

Viết chương trình xây dựng lớp Diem gồm:

-Các thuộc tính double x, y tương ứng là hai giá trị hoành độ và tung độ và các phương thức:

Các phương thức tạo lập:

```
public Diem(){ }
public Diem(double x,double y){x=x1;y=y1}
```

Các phương thức :

```
public void inTTDiem(){ // in giá trị x,y ra màn hình }
public double tinhKhoangCach(Diem d) {tính khoảng cách giữa điểm
hiện thời và đối số d}
```

Cài đặt lớp **Main** thực hiện tạo ra hai điểm $D1=new Diem(3,7); D2= new Diem(9,3)$ và in khoảng cách của hai điểm trên.

```

import java.util.Scanner;

public class Diem {
    double x,y;
    Diem() {
    }
    public void nhapTTDiem()
    {Scanner input= new Scanner(System.in);
    System.out.print("Nhap toa do x:= ");
    x=input.nextDouble();
    System.out.print("Nhap toa do y:= ");
    y=input.nextDouble();
    }
    public void inTTDiem()
    {System.out.println("Toa do X:"+x+"Toa do Y: "+y);
    }
    Diem(double x1,double y1)
    { x=x1;y=y1;
    }
    public double tinhKhoangCach(Diem d1)
    { return Math.sqrt((x-d1.x)*(x-d1.x)+(y-d1.y)*(y-d1.y));
    }
}

```

4.4. Các bài tập thực hành

Bài 1:

1. Xây dựng lớp PhanSo với hai thuộc tính riêng xác định tử số và mẫu số của phân số, đồng thời có các phương thức sau:
 - + Các toán tử tạo lập
 - + Các phép toán cộng, trừ, nhân, chia hai phân số
 - + Phép kiểm tra một phân số có phải tối giản hay không
 - + Phép tìm dạng tối giản của phân số
2. Viết một chương trình ứng dụng thực hiện việc nhập vào một dãy các phân số
 - + In ra màn hình dạng tối giản của các phân số đó

- + Sắp xếp các phân số theo thứ tự tăng dần của tử số của mỗi phân số

Bài 2: Để quản lý các hộ dân trong một khu phố, người ta quản lý các thông tin như sau:

- Với mỗi hộ dân, có các thuộc tính:
 - + Số thành viên trong hộ (số người)
 - + Số nhà của hộ dân đó (Số nhà được gán cho mỗi hộ dân)
 - + Thông tin về mỗi cá nhân trong hộ gia đình.
- Với mỗi cá nhân, người ta quản lý các thông tin như: họ và tên, tuổi, năm sinh, nghề nghiệp.

1. Hãy xây dựng các lớp: NhanSu để quản lý thông tin về mỗi cá nhân; lớp HoDan để quản lý thông tin về các hộ gia đình. Viết các phương thức để nhập, hiển thị thông tin cho mỗi cá nhân.

2. Cài đặt chương trình thực hiện các công việc sau:

- + Nhập vào một dãy gồm n hộ dân (n - nhập từ bàn phím).
- + Hiển thị ra màn hình thông tin về các hộ trong khu phố.

Bài 3: Một công ty được giao nhiệm vụ quản lý các phương tiện giao thông gồm các loại: ô tô, xe máy, xe tải.

- + Mỗi loại phương tiện giao thông cần quản lý: Số máy, hãng sản xuất, năm sản xuất, giá bán.
- + Các ô tô cần quản lý: số chỗ ngồi, kiểu động cơ
- + Xe máy cần quản lý: công suất
- + Xe tải cần quản lý: trọng tải.

1. Xây dựng các lớp XeTai, XeMay, OTo kế thừa từ lớp PTGT; đồng thời có các hàm để truy nhập, hiển thị và kiểm tra các thuộc tính của các lớp.

2. Xây dựng lớp QLPTGT cài đặt các phương thức thực hiện các chức năng sau:

- + Nhập thông tin đăng ký cho một danh sách gồm m phương tiện
- + Nhập vào số máy của một phương tiện, cho biết phương tiện đó thuộc

loại gì? Và hiển thị thông tin quản lý của phương tiện đó.

Bài 4: Một thư viện cần một hệ thống để quản lý sách, người mượn sách, và các giao dịch mượn trả sách. Hệ thống phải lưu trữ thông tin sách, người mượn, và theo dõi các giao dịch mượn trả.

Phân tích và giải quyết:

- **Xác định các lớp (Classes):**

- Book: Đại diện cho mỗi cuốn sách, chứa thông tin như mã sách, tên sách, tác giả, và số lượng có sẵn.
- Member: Đại diện cho mỗi người mượn, chứa thông tin như mã người mượn, tên, và số điện thoại.
- Transaction: Đại diện cho mỗi giao dịch mượn trả, chứa thông tin như mã giao dịch, người mượn, sách, ngày mượn, và ngày trả.

- **Xác định các thuộc tính và phương thức:**

- Book:
 - **Thuộc tính:** bookID, title, author, quantity.
 - **Phương thức:** updateQuantity().
- Member:
 - **Thuộc tính:** memberID, name, phone.
 - **Phương thức:** Không cần phương thức đặc biệt.
- Transaction:
 - **Thuộc tính:** transactionID, member, book, borrowDate, returnDate.
 - **Phương thức:** borrowBook(), returnBook().

- **Mối quan hệ giữa các lớp:**

- Transaction có mối quan hệ nhiều-nhiều với Book.
- Transaction có mối quan hệ 1-nhiều với Member.

Bài 5: Quản lý nhân sự

Mô tả: Một công ty cần hệ thống quản lý nhân sự để theo dõi thông tin của các nhân viên, bao gồm tên, mã nhân viên, chức vụ, và lương. Hệ thống cũng cần có khả năng tính toán lương tháng dựa trên các yếu tố như ngày làm việc, số giờ làm thêm và các khoản thưởng phạt.

Gợi ý giải quyết:

- **Lớp:** Employee (Nhân viên)

- **Thuộc tính:** employeeID, name, position, salary, workDays, overtimeHours, bonus, penalty.
- **Phương thức:** calculateMonthlySalary(), addWorkDays(), addOvertimeHours().

Bài 6: Quản lý tài khoản ngân hàng

Mô tả: Ngân hàng cần một hệ thống để quản lý tài khoản của khách hàng, bao gồm việc mở tài khoản, nạp tiền, rút tiền, và kiểm tra số dư. Mỗi khách hàng có thể có nhiều tài khoản với số dư khác nhau.

Gợi ý giải quyết:

- **Lớp:** BankAccount (Tài khoản ngân hàng), Customer (Khách hàng).
- **Thuộc tính:** accountNumber, balance, accountType cho BankAccount; customerID, name, accounts cho Customer.
- **Phương thức:** deposit(), withdraw(), checkBalance() cho BankAccount; addAccount() cho Customer.

Bài 7: Quản lý cửa hàng sách

Mô tả: Một cửa hàng sách cần hệ thống để quản lý sách, tác giả, và đơn hàng. Hệ thống cần lưu trữ thông tin sách, theo dõi số lượng sách bán ra, và quản lý đơn hàng của khách hàng.

Gợi ý giải quyết:

- **Lớp:** Book (Sách), Author (Tác giả), Order (Đơn hàng).
- **Thuộc tính:** bookID, title, author, price, stock cho Book; authorID, name, booksWritten cho Author; orderID, customer, booksOrdered cho Order.
- **Phương thức:** sellBook(), updateStock() cho Book; addBook() cho Author; calculateTotalPrice() cho Order.

Bài 8: Quản lý hệ thống đặt phòng khách sạn

Mô tả: Một khách sạn cần một hệ thống để quản lý đặt phòng, khách hàng và thanh toán. Hệ thống phải có khả năng quản lý các phòng trống, kiểm tra tình trạng phòng, và xử lý đặt phòng cũng như thanh toán.

Gợi ý giải quyết:

- **Lớp:** Room (Phòng), Customer (Khách hàng), Booking (Đặt phòng).
- **Thuộc tính:** roomID, roomType, price, availability cho Room; customerID, name, phone cho Customer; bookingID, room, customer, checkInDate, checkOutDate cho Booking.
- **Phương thức:** checkAvailability(), bookRoom() cho Room; makePayment() cho Booking.

Bài 9: Quản lý hệ thống học trực tuyến

Mô tả: Một tổ chức giáo dục cần một hệ thống để quản lý các khóa học trực tuyến, học viên và giảng viên. Hệ thống cần lưu trữ thông tin về khóa học, danh sách học viên đăng ký và điểm số của họ.

Gợi ý giải quyết:

- **Lớp:** Course (Khóa học), Student (Học viên), Instructor (Giảng viên).
- **Thuộc tính:** courseID, courseName, instructor, students cho Course; studentID, name, coursesEnrolled cho Student; instructorID, name, coursesTaught cho Instructor.
- **Phương thức:** enrollStudent(), assignInstructor(), recordGrade() cho Course; registerCourse() cho Student.